

Desktop GUI & Client Shell — Dependency Structure Matrix & Propagation Cost

Sub-team 6 · Quality Champion · BNCH-4 · Day 4 · 21 May 2026

1. Scope

This report presents the Dependency Structure Matrix (DSM), fan-in/fan-out analysis, propagation cost, and dependency cycle audit for the eight classes owned by the Desktop GUI and Client Shell work package. The analysis feeds deliverable T2 (modularity assessment) and the competitive pitch.

2. Dependency Structure Matrix

Row depends on column. **X** = direct source-level dependency within the 8-class subsystem.

[illegible]

Ext. CBO = project-specific classes outside the 8 (e.g. VolumeDataSetRenderer, VolumeInputController, FitsReader, DataAnalysis, OxyPlot types). Unity framework types excluded.

3. Fan-In, Fan-Out, and Propagation Cost

Propagation cost = percentage of all 8 classes affected (directly or transitively) when this class changes.

Class	Fan-In	Fan-Out	Propagation Cost
CanvassDesktop	4	4	75% (6/8) 
PaintMenuController	2	0	87.5% (7/8) 
HistogramMenuController	1	1	75% (6/8) 
HistogramHelper	2	2	75% (6/8) 
TabsManager	1	1	75% (6/8) 
SourceRow	1	1	75% (6/8) 
DesktopPaintController	0	2	12.5% (1/8) 
VideoUiManager	0	0	12.5% (1/8) 

Propagation cost derivation (PaintMenuController example): PMC is depended on by CD and DPC. Via CD the change cascades to HH, TM, SR. Via HH it reaches HMC. Total affected: PMC + CD + DPC + HH + TM + SR + HMC = 7 of 8 classes.

Key observation: 6 of 8 classes have propagation costs $\geq 75\%$. Only VideoUiManager (fully decoupled) and DesktopPaintController (leaf node) are safe to change in isolation. The subsystem is effectively monolithic despite being split across 8 files.

4. Dependency Cycles (Architectural Violations)

Three bilateral dependency cycles exist, all violating the mandatory constraint in Section 4.2: **"No circular dependencies between top-level components. Cyclic package dependencies are a fail condition."**

Cycle 1: CanvassDesktop $\leftrightarrow$ TabsManager

Evidence: CanvassDesktop holds `[SerializeField] TabsManager _tabsManager`. TabsManager holds `[SerializeField] CanvassDesktop _canvassDesktop` and calls `_canvassDesktop.paintTabSelected() / paintTabLeft()`.

Remediation: Introduce an `Action paintTabSelected` event/delegate on CanvassDesktop that TabsManager subscribes to, removing TabsManager's direct reference to CanvassDesktop.

Cycle 2: CanvassDesktop ↔ HistogramHelper

Evidence: CanvassDesktop references HistogramHelper and calls `UpdateUI()`. HistogramHelper holds `public CanvassDesktop canvassDesktop` and calls `canvassDesktop.UpdateUI()`.

Remediation: Extract an `IHistogramDisplay` interface. CanvassDesktop implements it. HistogramHelper depends on the interface, not the concrete class.

Cycle 3: HistogramHelper ↔ HistogramMenuController

Evidence: HistogramHelper calls `histogramMenu.UpdateUI()` on HistogramMenuController. HistogramMenuController holds a reference back to HistogramHelper.

Remediation: Same `IHistogramDisplay` interface from Cycle 2 also decouples this — HistogramHelper calls the interface rather than either concrete class. Cycles 2 and 3 collapse in one move.

Cycle Cluster

Cycles 2 and 3 together mean CanvassDesktop, HistogramHelper, and HistogramMenuController form a **mutually entangled cluster** — a change anywhere in the trio ripples through all three.

5. Highest-Risk Change Target

PaintMenuController carries the highest propagation cost (87.5%) despite having zero internal fan-out. This is because its two dependents (CanvassDesktop and DesktopPaintController) sit upstream of the entire dependency web. PMC is the lowest-effort entry point to trigger a cascade across the subsystem.

CanvassDesktop is the highest structural risk overall. It has equal-highest fan-in and fan-out (both 4), participates in 2 of the 3 cycles, spans ~1,900 lines of mixed concerns, and carries 13 external project-class dependencies. Any refactor touching CanvassDesktop requires coordinated changes in DPC, HH, TM, and SR simultaneously.

Recommended Remediation Priority

1. **Break Cycle 1 (CD ↔ TM):** Event/delegate pattern — TabsManager subscribes to CanvassDesktop events instead of holding a direct reference.
2. **Break Cycles 2 + 3 (CD ↔ HH ↔ HMC):** Extract `IHistogramDisplay` interface — both CanvassDesktop and HistogramMenuController implement it, HistogramHelper depends on the interface. This collapses both cycles in one refactoring move.
3. **Decompose CanvassDesktop:** Split into focused controllers (file loading, source table, UI state) to reduce fan-in and external CBO before further cycle remediation.